

Quick Sort

Section 5

Quick Sort

Definition: A divide-and-conquer sorting algorithm that works by selecting a pivot and partitioning elements into two subarrays:

- Elements smaller than or equal the pivot
- Elements greater than the pivot

Algorithm Steps

1. Pick a pivot element.

2. Partition the array into two parts:
    - Left: elements <= pivot
    - Right: elements > pivot

3. Recursively apply the same steps to both parts.

Visualization & Pseudocode

Visual Sort here

```
QUICKSORT(arr, start, end):
    if start >= end:
        return

    pivotIndex = PARTITION(arr, start, end)

    QUICKSORT(arr, start, pivotIndex - 1)   // Left side
    QUICKSORT(arr, pivotIndex + 1, end)     // Right side
```

```
PARTITION(arr, start, end):
    pivot = arr[start]
    nxt = start + 1

    for i from start + 1 to end:
        if arr[i] <= pivot:
            swap arr[i] and arr[nxt]
            nxt = nxt + 1

    swap arr[start] and arr[nxt - 1]

    return nxt - 1    // New pivot position
```

Code is here

What You Must Do

- Study the Quick Sort code carefully
- Understand how the pivot is chosen and how the partition works
- Trace the algorithm step-by-step for small examples
- Write the code yourself (not copy-paste)
- Modify the code, such as:
    - Change pivot selection (last element or random)
    - Print the array after each partition
    - Sort in descending order
- Make sure you fully understand each recursion call

Analysis of Quick Sort

Quick Sort Complexity (Summary)

- Best Case: O(n log n) — pivot evenly splits the array
- Average Case: O(n log n) — random pivot gives balanced partitions
- Worst Case: $O(n^2)$ — pivot repeatedly chooses smallest/largest element

Partition Example

Illustrate the operation of PARTITION on the array:

A = ⟨13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11⟩.

Solution

We move through the array and swap any value ≤ **pivot** with the element at `nxt`, then increment `nxt`.

i	A[i]	Compare with 13	Action	Updated Array	nxt
1	19	> 13	Do nothing	⟨13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11⟩	1
2	9	≤ 13	swap A[2] ↔ A[1]	⟨13, **9**, 19, 5, 12, 8, 7, 4, 21, 2, 6, 11⟩	2
3	5	≤ 13	swap A[3] ↔ A[2]	⟨13, 9, **5**, 19, 12, 8, 7, 4, 21, 2, 6, 11⟩	3
4	12	≤ 13	swap A[4] ↔ A[3]	⟨13, 9, 5, **12**, 19, 8, 7, 4, 21, 2, 6, 11⟩	4
5	8	≤ 13	swap	⟨13, 9, 5, 12, **8**, 19, 7, 4, 21, 2, 6, 11⟩	5
6	7	≤ 13	swap	⟨13, 9, 5, 12, 8, **7**, 19, 4, 21, 2, 6, 11⟩	6
7	4	≤ 13	swap	⟨13, 9, 5, 12, 8, 7, **4**, 19, 21, 2, 6, 11⟩	7
8	21	> 13	Do nothing	—	7
9	2	≤ 13	swap	⟨13, 9, 5, 12, 8, 7, 4, **2**, 21, 19, 6, 11⟩	8
10	6	≤ 13	swap	⟨13, 9, 5, 12, 8, 7, 4, 2, **6**, 19, 21, 11⟩	9
11	11	≤ 13	swap	⟨13, 9, 5, 12, 8, 7, 4, 2, 6, **11**, 21, 19⟩	10

Quick sort Example

Illustrate the operation of Quicksort sort on an array initially containing the sequence {3; 41; 52; 26; 38; 57; 9; 49}

Solution

```
// Initial array
3 41 52 26 38 57 9 49

// Partition with pivot = 3
3 41 52 26 38 57 9 49

// Partition right subarray [41 52 26 38 57 9 49], pivot = 41
3 26 38 9 41 52 57 49

// Partition left of pivot 41 → [26 38 9], pivot = 26
3 9 26 38 41 52 57 49

// 9 is a single element (sorted)
// 38 is a single element (sorted)

// Partition right of pivot 41 → [52 57 49], pivot = 52
3 9 26 38 41 49 52 57

// 49 is a single element (sorted)
// 57 is a single element (sorted)

// Final sorted array
3 9 26 38 41 49 52 57
```

Solve this

Illustrate the operation of Quicksort sort on the array A = ⟨13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11⟩.



Now it’s time to run the code yourself and modify it to sort in descending order.